Evaluating Role-Structured Multi-Agent LLM Architectures for Diagnostic Classification Using Structured Clinical Data

Callum Anderson MIAI, University of Ottawa cande007@uottawa.ca

May 10, 2026

Outline

1 Purpose 1

2 Primary Research Question 1

3 Preprocessing UCI Cleveland 2

4 Dataset Feature Schema (UCI Heart Disease – Processed Cleveland Subset) 2

5 Preprocessing PIMA Indians 3

6 Dataset Feature Schema (PIMA Indians) 3

7 Data sanity check 4

8 Heart Disease Patient Prompt Serialization 5

9 Diabetes Patient Prompt Serialization 5

10 Diagnostic Task Definition 5

11 Evaluation Metrics 6

12 Shared Utility Code 6

13 Single-Agent Baseline 7
- 13.1 Pipeline . . . 7
- 13.2 Inference . . . 8
- 13.3 Results . . . 8

14 Generic Deliberative Protocol 8
- 14.1 Pipeline . . . 8
- 14.2 Inference . . . 11
- 14.3 Results . . . 11

15 Feature-Specialist Protocol **12**
15.1 Pipeline . . . 12
15.2 Run . . . 14
15.3 Results . . . 15

16 Results Analysis **15**
16.1 Overall Performance (Cleveland) . . . 15
16.2 Overall Performance (Pima Indians Diabetes) . . . 15
16.3 Class-wise Performance (Cleveland) . . . 16
16.4 Class-wise Performance (Pima Indians Diabetes) . . . 16
16.5 Confusion Matrices . . . 16

17 Ablation Study: Feature-Set Sensitivity of the Feature-Specialist Protocol (Cleveland) **17**
17.1 Motivation . . . 17
17.2 Feature Conditions and Rationale . . . 18
17.3 Ablation Results Summary . . . 23
17.4 Ablation Class-wise Performance Summary . . . 23

18 Ablation Study: Feature-Set Sensitivity of the Feature-Specialist Protocol (Pima Indians Diabetes) **24**
18.1 Motivation . . . 24
18.2 Feature Conditions and Rationale . . . 24
18.3 Ablation Results Summary . . . 29
18.4 Ablation Class-wise Performance Summary . . . 29

19 Ablation Study: Model Scale — Qwen2.5 14B (Original GD and FS Configurations) **30**
19.1 Motivation . . . 30
19.2 Model-Scale Ablation Results (qwen2.5:14b) . . . 33
19.3 Overall Performance (Cleveland) . . . 33
19.4 Overall Performance (Pima Indians Diabetes) . . . 33
19.5 Class-wise Performance (Cleveland) . . . 34
19.6 Class-wise Performance (Pima Indians Diabetes) . . . 34

1 Purpose

This notebook implements and evaluates two **role-structured multi-agent LLM protocols** on two different tabular clinical diagnostic tasks.

This study investigates how **internal agent role structure** influences classification behavior under controlled conditions.

The two protocols evaluated are:

- **Generic Deliberative Protocol** Parallel generalist "doctor" agents independently review the full patient record, followed by a final adjudicator.
- **Feature-Specialist Protocol** Parallel specialist agents evaluate distinct clinical features and submit structured opinions to a final aggregation judge.

Both protocols: - Use the **same base LLM** - Use **deterministic decoding (temperature = 0)** - Use identical structured JSON output constraints - Process the same serialized tabular input - Use equal numbers of inference calls per patient

This study does **not** aim to outperform classical machine learning baselines. Instead, it isolates how **agent role decomposition acts as an architectural inductive bias** in LLM-based classification.

---

2 Primary Research Question

Can different role-conditioned multi-agent prompting protocols systematically alter the error profile and operating characteristics of LLM-based clinical classifiers under identical deterministic conditions?

```
from __future__ import annotations

from IPython.display import display
import pandas as pd
from langchain_core.prompts import ChatPromptTemplate
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
import numpy as np
import json
import re
from dataclasses import dataclass
from typing import Any, Dict, Optional
from langchain_ollama import ChatOllama
```

```
MODEL_NAME = "llama3.1:8b"   # change
TEMPERATURE = 0.0
LABELS = ["no_disease", "disease_present"]
df = pd.read_csv("data/processed_diabetes.csv")
OUT_CSV_SINGLE = "single-base-predictions.csv"
OUT_CSV_GENERIC = "generic-predictions.csv"
OUT_CSV_SPECIALIST = "specialist-predictions.csv"
# FEATURE_1 = "chest_pain_type" <- heart_disease specialist feature 1
# FEATURE_2 = "num_major_vessels_fluoroscopy" <- heart_disease specialist feature 2
FEATURE_1 = "plasma_glucose_mg_dL"
FEATURE_2 = "body_mass_index"
```

3 Preprocessing UCI Cleveland

Given the numeric format of the data, I will need to rename the features as well as map their numerical values to text values in order to utilize LLMs effectively.

Missing values for categorical features are imputed with mode.

Below is that mapping:

4 Dataset Feature Schema (UCI Heart Disease – Processed Cleveland Subset)

#	Original Index	Raw Name	Description	Encoded Values / Units	LLM-Friendly Feature Name
1	#3	age	Age of the patient	Years	`age_years`
2	#4	sex	Biological sex	1 = male, 0 = female	`sex`

#	Original Index	Raw Name	Description	Encoded Values / Units	LLM-Friendly Feature Name
3	#9	cp	Chest pain type	1 = typical angina2 = atypical angina3 = non-anginal pain4 = asymptomatic	`chest_pain_type`
4	#10	trestbps	Resting blood pressure	mm Hg	`resting_blood_pressure_mmHg`
5	#12	chol	Serum cholesterol	mg/dL	`serum_cholesterol_mg_dL`
6	#16	fbs	Fasting blood sugar > 120 mg/dL	1 = true, 0 = false	`fasting_blood_sugar_over_120_mg_dL`
7	#19	restecg	Resting electrocardiographic results	0 = normal1 = ST-T wave abnormality2 = left ventricular hypertrophy	`resting_ecg_result`
8	#32	thalach	Maximum heart rate achieved	Beats per minute	`max_heart_rate_bpm`
9	#38	exang	Exercise-induced angina	1 = yes, 0 = no	`exercise_induced_angina`
10	#40	oldpeak	ST depression induced by exercise relative to rest	Numeric (float)	`st_depression_exercise_relative_to_rest`
11	#41	slope	Slope of peak exercise ST segment	1 = upsloping2 = flat3 = downsloping	`st_segment_slope_peak_exercise`
12	#44	ca	Number of major vessels colored by fluoroscopy	Integer (0–3)	`num_major_vessels_fluoroscopy`
13	#51	thal	Thalassemia test result	3 = normal6 = fixed defect7 = reversible defect	`thalassemia_test_result`
14	#58	num	Diagnosis of heart disease (target variable)	0 = no disease1–4 = disease present (increasing severity)	`disease_present` *(binary derived)*

5 Preprocessing PIMA Indians

The Pima Indians Diabetes dataset consists entirely of numeric clinical measurements recorded at a single visit. Several attributes (Glucose, BloodPressure, SkinThickness, Insulin, and BMI) contain physiologically implausible zero values that are conventionally treated as missing data and therefore require explicit handling prior to model evaluation. For LLM-based classification, features are serialized into structured textual form with units preserved to maintain clinical interpretability, while imputation is performed deterministically to ensure reproducibility across prompting protocols.

6 Dataset Feature Schema (PIMA Indians)

#	Original Index	Raw Name	Description	Encoded Values / Units	LLM-Friendly Feature Name
1	—	Pregnancies	Number of pregnancies	Integer (count)	pregnancies

#	Original Index	Raw Name	Description	Encoded Values / Units	LLM-Friendly Feature Name
2	—	Glucose	Plasma glucose concentration (2-hour OGTT)	mg/dL (0 commonly used as missing in distributed versions)	plasma_glucose_mg_dL
3	—	BloodPressure	Diastolic blood pressure	mm Hg (0 commonly used as missing in distributed versions)	diastolic_blood_pressure_mmHg
4	—	SkinThickness	Triceps skinfold thickness	mm (0 commonly used as missing in distributed versions)	triceps_skinfold_thickness_mm
5	—	Insulin	2-hour serum insulin	μU/mL (0 commonly used as missing in distributed versions)	serum_insulin_uU_mL
6	—	BMI	Body mass index	kg/m² (0 commonly used as missing in distributed versions)	body_mass_index
7	—	DiabetesPedigreeFunction	Diabetes pedigree function (family history score)	Numeric (unitless)	diabetes_pedigree_function
8	—	Age	Age of the patient	Years	age_years
9	—	Outcome	Diabetes diagnosis (target variable)	0 = no diabetes, 1 = diabetes present (binary native label)	disease_present (binary)

Notes:

- All predictors are numeric.
- Each row represents a single patient at a single clinical visit.
- Glucose, BloodPressure, SkinThickness, Insulin, and BMI often include physiologically implausible zeros in public releases; these typically represent missing values and should be documented in preprocessing.

7 Data sanity check

Perform some basic EDA.

I also impute missing or null values from the original data with that columns mode.

```
df.shape
```

```
df["disease_present"].value_counts(normalize=True)
```

```
OUT_CSV_CLEAN = "data/processed_diabetes.csv"
MISSING_TOKENS = ["", "?", "null", "NULL", "none", "None", "NA", "N/A", "nan", "NaN"]
df = df.replace(MISSING_TOKENS, np.nan)

obj_cols = df.select_dtypes(include=["object", "string"]).columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip()).replace("", np.nan)

for col in df.columns:
    if df[col].isna().any():
        mode_val = df[col].mode(dropna=True)
```

```
        if not mode_val.empty:
            df[col] = df[col].fillna(mode_val.iloc[0])

df.to_csv(OUT_CSV_CLEAN, index=False)
df_check = pd.read_csv("data/processed_diabetes.csv")
print((df_check.isna() | (df_check == "") | (df_check == "?")).sum())
```

8 Heart Disease Patient Prompt Serialization

This section describes how structured patient records from the UCI Cleveland dataset are transformed into natural-language prompts suitable for large language model inference.

```
"""
def serialize_patient(row):
    return f"""
# Patient information:
# - Age: {row['age_years']}
# - Sex: {row['sex']}
# - Chest pain type: {row['chest_pain_type']}
# - Resting blood pressure: {row['resting_blood_pressure_mmHg']} mmHg
# - Serum cholesterol: {row['serum_cholesterol_mg_dL']} mg/dL
# - Fasting blood sugar > 120 mg/dL: {row['fasting_blood_sugar_over_120_mg_dL']}
# - Resting ECG result: {row['resting_ecg_result']}
# - Maximum heart rate achieved: {row['max_heart_rate_bpm']} bpm
# - Exercise-induced angina: {row['exercise_induced_angina']}
# - ST depression induced by exercise: {row['st_depression_exercise_relative_to_rest']}
# - ST segment slope during peak exercise: {row['st_segment_slope_peak_exercise']}
# - Number of major vessels (fluoroscopy): {row['num_major_vessels_fluoroscopy']}
# - Thalassemia test result: {row['thalassemia_test_result']}
"""

print(serialize_patient(df.iloc[0])) """
```

9 Diabetes Patient Prompt Serialization

This section describes how structured patient records from the pima indians dataset are transformed into natural-language prompts suitable for large language model inference.

```
def serialize_patient(row):
    return f"""
Patient information:
- Age: {row['age_years']}
- Pregnancies: {row['pregnancies']}
- Plasma glucose concentration (2-hour OGTT): {row['plasma_glucose_mg_dL']} mg/dL
- Diastolic blood pressure: {row['diastolic_blood_pressure_mmHg']} mmHg
- Triceps skinfold thickness: {row['triceps_skinfold_thickness_mm']} mm
- 2-hour serum insulin: {row['serum_insulin_uU_mL']} uU/mL
- Body mass index (BMI): {row['body_mass_index']} kg/m^2
- Diabetes pedigree function: {row['diabetes_pedigree_function']}
"""

print(serialize_patient(df.iloc[0]))
```

10 Diagnostic Task Definition

The primary task is binary classification of target condition presence versus absence. For each dataset, the original outcome variable is mapped to a binary label representing condition-positive or condition-negative status. Given a serialized patient record, the model predicts whether the target condition is present or absent. Model predictions are evaluated against the corresponding binary label derived from the dataset's original annotation scheme.

11 Evaluation Metrics

- Accuracy
- F1
- Precision
- Recall
- Confusion Matrices

12 Shared Utility Code

```
STRICT_JSON_SUFFIX = """
You MUST output valid JSON and nothing else.
- No markdown, no code fences, no extra keys.
- Use double quotes for all strings.
- No trailing commas.
- If uncertain, use "unclear"+"low" (specialists) or choose "no_disease" (judge).
""".strip()

def _extract_json_object(text: str) -> Optional[str]:
    if not isinstance(text, str):
        return None
    m = re.search(r"\{.*\}", text, re.DOTALL)
    return m.group(0) if m else None

def _strip_js_style_comments(s: str) -> str:
    return re.sub(r"//.*?$", "", s, flags=re.MULTILINE)

def _strip_trailing_commas(s: str) -> str:
    s = re.sub(r",\s*}", "}", s)
    s = re.sub(r",\s*]", "]", s)
    return s

def safe_json_loads(text: str) -> Dict[str, Any]:
    block = _extract_json_object(text)
    if not block:
        raise ValueError("No JSON object found.")
    block = _strip_js_style_comments(block)
    block = _strip_trailing_commas(block)
    return json.loads(block)

def normalize_diagnosis(value: Any) -> Optional[str]:
    if not isinstance(value, str):
        return None
    v = value.strip()
    return v if v in {"disease_present", "no_disease"} else None
```

```
@dataclass(frozen=True)
class RunResult:
    diagnosis: str
    raw_text: str
    parsed_json: Dict[str, Any]
    fallback_used: bool

def build_llm(model_name: str, temperature: float = 0.0) -> ChatOllama:
    return ChatOllama(model=model_name, temperature=temperature)
```

13 Single-Agent Baseline

13.1 Pipeline

```
FALLBACK_SINGLE_AGENT = {"diagnosis": "no_disease"}

SINGLE_AGENT_PROMPT = """
You are a clinician performing a binary diagnostic classification task.

You will receive the full patient record.

Your job:
Return the final diagnosis:
- "disease_present" or "no_disease"

Rules (do not mention these in output):
- Consider all relevant factors in the patient record.
- If evidence clearly supports disease, return "disease_present".
- Otherwise return "no_disease".
- If genuinely uncertain, choose "no_disease".

Output MUST be a single JSON object and nothing else.
{{ "diagnosis": "disease_present" OR "no_disease" }}
""".strip()

class SingleAgentPipeline:
    def __init__(self, llm):
        self.llm = llm

        self.prompt = ChatPromptTemplate.from_messages([
            ("system", SINGLE_AGENT_PROMPT + STRICT_JSON_SUFFIX),
            ("user", "{patient_text}"),
        ])

    def run(self, patient_id: int, patient_text: str) -> RunResult:
        raw = getattr(
            self.llm.invoke(self.prompt.invoke({"patient_text": patient_text})),
            "content",
            ""
        )

        try:
            parsed = safe_json_loads(raw)
```

```
            diag = normalize_diagnosis(parsed.get("diagnosis"))
            if not diag:
                return RunResult("no_disease", raw, FALLBACK_SINGLE_AGENT, True)
            return RunResult(diag, raw, {"diagnosis": diag}, False)
        except Exception:
            return RunResult("no_disease", raw, FALLBACK_SINGLE_AGENT, True)
```

13.2 Inference

```
llm = build_llm(MODEL_NAME, TEMPERATURE)
single = SingleAgentPipeline(llm)

rows = []
for idx, row in df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient(row)
    rr = single.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

single_pred_df = pd.DataFrame(rows)
single_pred_df.to_csv(OUT_CSV_SINGLE, index=False)
single_pred_df.head()
```

13.3 Results

```
LABELS = ["no_disease", "disease_present"]

eval_df = (
    df.reset_index(drop=True)
      .reset_index(names="row_index")
      .merge(single_pred_df[["row_index", "prediction"]],
             on="row_index",
             how="inner")
)

eval_df = eval_df[
    eval_df["disease_present"].isin(LABELS) &
    eval_df["prediction"].isin(LABELS)
].copy()

y_true = eval_df["disease_present"]
y_pred = eval_df["prediction"]

print("=" * 90)
print("Single-Agent Protocol — classification_report")
print(classification_report(
    y_true,
    y_pred,
    labels=LABELS,
    target_names=LABELS,
    digits=2
))
```

14 Generic Deliberative Protocol

14.1 Pipeline

```
FALLBACK_DOCTOR_OPINION = {"scope": "overall_record", "signal": "unclear", "strength": "low", "note":
     ""}
FALLBACK_JUDGE          = {"diagnosis": "no_disease"}

GENERIC_DOCTOR_OPINION_PROMPT = """
You are a clinician reviewing the FULL patient record.

Task:
Provide an overall opinion about whether the FULL RECORD supports:
- supports_disease
- supports_no_disease
- unclear

Guidelines (do not mention these in output):
- Use "supports_disease" only if overall evidence clearly supports disease.
- Use "supports_no_disease" only if overall evidence clearly supports no disease.
- Otherwise use "unclear".
- "strength" reflects how strong the overall evidence is (high > medium > low).
- "note" must be ONE short sentence citing —12 key factors from the record (no long reasoning).

Output MUST be a single JSON object and nothing else:
{{
  "scope": "overall_record",
  "signal": "supports_disease" OR "supports_no_disease" OR "unclear",
  "strength": "low" OR "medium" OR "high",
  "note": "one short sentence"
}}
""".strip()

GENERIC_ADJUDICATOR_PROMPT = """
You are an adjudicator for a binary clinical classification task.

You will receive:
1) The full patient record
2) Two doctor opinions about the full record (signal + strength)

Your job:
Return the final diagnosis:
- "disease_present" or "no_disease"

Rules (do not mention these in output):
- Treat each doctor opinion as evidence about the overall record; it may be low quality or wrong.
- Weigh evidence by "strength" (high > medium > low).
- If both doctors strongly support the same direction, follow that direction.
- If evidence conflicts or is weak/unclear, use the full patient record to decide.
- If still genuinely uncertain after considering the full record, choose "no_disease" as a tie-break.
- Do not automatically prefer Doctor 1 or Doctor 2.

Output MUST be a single JSON object and nothing else.
{{ "diagnosis": "disease_present" OR "no_disease" }}
""".strip()
```

```python
class GenericDeliberativeMASPipeline:
    def __init__(self, llm):
        self.llm = llm

        # doctor 1 (overall opinion)
        self.doc1_prompt = ChatPromptTemplate.from_messages([
            ("system", GENERIC_DOCTOR_OPINION_PROMPT + STRICT_JSON_SUFFIX),
            ("user", "{patient_text}"),
        ])

        # doctor 2 (overall opinion)
        self.doc2_prompt = ChatPromptTemplate.from_messages([
            ("system", GENERIC_DOCTOR_OPINION_PROMPT + STRICT_JSON_SUFFIX),
            ("user", "{patient_text}"),
        ])

        # adjudicator
        self.judge_prompt = ChatPromptTemplate.from_messages([
            ("system", GENERIC_ADJUDICATOR_PROMPT + STRICT_JSON_SUFFIX),
            ("user",
             "Patient information:\n{patient_text}\n\n"
             "Doctor opinions:\n"
             "- {op1}\n"
             "- {op2}\n"),
        ])

    def run(self, patient_id: int, patient_text: str) -> RunResult:
        any_fb = False

        # doctor 1
        raw1 = getattr(
            self.llm.invoke(self.doc1_prompt.invoke({"patient_text": patient_text})),
            "content",
            ""
        )
        try:
            op1 = safe_json_loads(raw1)
        except Exception:
            op1 = dict(FALLBACK_DOCTOR_OPINION)
            any_fb = True

        if not isinstance(op1, dict):
            op1 = dict(FALLBACK_DOCTOR_OPINION)
            any_fb = True
        op1.setdefault("scope", "overall_record")
        op1.setdefault("signal", "unclear")
        op1.setdefault("strength", "low")
        op1.setdefault("note", "")

        # doctor 2
        raw2 = getattr(
            self.llm.invoke(self.doc2_prompt.invoke({"patient_text": patient_text})),
            "content",
            ""
        )
```

```
        try:
            op2 = safe_json_loads(raw2)
        except Exception:
            op2 = dict(FALLBACK_DOCTOR_OPINION)
            any_fb = True

        if not isinstance(op2, dict):
            op2 = dict(FALLBACK_DOCTOR_OPINION)
            any_fb = True
        op2.setdefault("scope", "overall_record")
        op2.setdefault("signal", "unclear")
        op2.setdefault("strength", "low")
        op2.setdefault("note", "")

        # adjudicator
        rawj = getattr(
            self.llm.invoke(self.judge_prompt.invoke({
                "patient_text": patient_text,
                "op1": json.dumps(op1, ensure_ascii=False),
                "op2": json.dumps(op2, ensure_ascii=False),
            })),
            "content",
            ""
        )

        try:
            parsed = safe_json_loads(rawj)
            diag = normalize_diagnosis(parsed.get("diagnosis"))
            if not diag:
                return RunResult("no_disease", rawj, FALLBACK_JUDGE, True)
            return RunResult(diag, rawj, {"diagnosis": diag}, any_fb)
        except Exception:
            return RunResult("no_disease", rawj, FALLBACK_JUDGE, True)
```

14.2 Inference

```
llm = build_llm(MODEL_NAME, TEMPERATURE)
mas = GenericDeliberativeMASPipeline(llm)

rows = []
for idx, row in df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient(row)
    rr = mas.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

generic_pred_df = pd.DataFrame(rows)
generic_pred_df.to_csv(OUT_CSV_GENERIC, index=False)
generic_pred_df.head()
```

14.3 Results

```
LABELS = ["no_disease", "disease_present"]

eval_df = (
```

```
    df.reset_index(drop=True)
      .reset_index(names="row_index")
      .merge(generic_pred_df[["row_index", "prediction"]],
             on="row_index",
             how="inner")
)

eval_df = eval_df[
    eval_df["disease_present"].isin(LABELS) &
    eval_df["prediction"].isin(LABELS)
].copy()

y_true = eval_df["disease_present"]
y_pred = eval_df["prediction"]

print("=" * 90)
print("Generic Deliberative Protocol — classification_report")
print(classification_report(
    y_true,
    y_pred,
    labels=LABELS,
    target_names=LABELS,
    digits=2
))
```

15 Feature-Specialist Protocol

15.1 Pipeline

```
FALLBACK_SPECIALIST = {"feature": "", "signal": "unclear", "strength": "low", "note": ""}
FALLBACK_JUDGE      = {"diagnosis": "no_disease"}

SPECIALIST_OPINION_TEMPLATE = """
You are a clinical feature specialist. You ONLY evaluate the feature: "{feature_name}".

Task:
Given the patient record, decide whether THIS FEATURE ALONE:
- supports_disease
- supports_no_disease
- unclear

Guidelines (do not mention these in output):
- Use "supports_disease" only if this feature value is a meaningful risk indicator by itself.
- Use "supports_no_disease" only if this feature value is meaningfully reassuring by itself.
- Otherwise use "unclear".
- "strength" reflects how strongly this single feature points in that direction (not overall
    diagnosis):
  - high: strong / clear signal
  - medium: moderate signal
  - low: weak, borderline, missing, or noisy
- "note" must be ONE short sentence that explicitly cites the 'patients "{feature_name}" value and
    why it points that way.
- Do NOT use any other features in your reasoning.
```

```
Output MUST be a single JSON object and nothing else, with EXACTLY these keys:
{{{{
  "feature": "{feature_name}",
  "signal": "supports_disease" OR "supports_no_disease" OR "unclear",
  "strength": "low" OR "medium" OR "high",
  "note": "one short sentence referencing the patient's {feature_name} value"
}}}}
""".strip()

JUDGE_PROMPT = """
You are an adjudicator for a binary clinical classification task.

You will receive:
1) The full patient record
2) Two specialist opinions (each about ONE feature only)

Your job:
Return the final diagnosis:
- "disease_present" or "no_disease"

Rules (do not mention these in output):
- Treat each specialist opinion as evidence about a single feature; it may be low quality or wrong.
- Weigh evidence by "strength" (high > medium > low).
- If both specialists strongly support the same direction, follow that direction.
- If evidence conflicts or is weak/unclear, use the full patient record to decide.
- If still genuinely uncertain after considering the full record, choose "no_disease" as a tie-break.

Output MUST be a single JSON object and nothing else.
{{ "diagnosis": "disease_present" OR "no_disease" }}
""".strip()


class TwoSpecialistsMASPipeline:
    def __init__(self, llm, feature_1: str, feature_2: str):
        self.llm = llm
        self.f1 = feature_1
        self.f2 = feature_2

        # specialist 1
        self.doc1_prompt = ChatPromptTemplate.from_messages([
            ("system", SPECIALIST_OPINION_TEMPLATE.format(feature_name=self.f1) + STRICT_JSON_SUFFIX)
    ,
            ("user", "{patient_text}"),
        ])

        # specialist 2
        self.doc2_prompt = ChatPromptTemplate.from_messages([
            ("system", SPECIALIST_OPINION_TEMPLATE.format(feature_name=self.f2) + STRICT_JSON_SUFFIX)
    ,
            ("user", "{patient_text}"),
        ])

        # judge/adjudicator
        self.judge_prompt = ChatPromptTemplate.from_messages([
            ("system", JUDGE_PROMPT + STRICT_JSON_SUFFIX),
            ("user",
```

```
            "Patient information:\n{patient_text}\n\n"
            "Specialist opinions:\n"
            "- {op1}\n"
            "- {op2}\n"),
        ])

    def run(self, patient_id: int, patient_text: str) -> RunResult:
        any_fb = False

        # specialist 1
        raw1 = getattr(
            self.llm.invoke(self.doc1_prompt.invoke({"patient_text": patient_text})),
            "content",
            ""
        )
        try:
            op1 = safe_json_loads(raw1)
        except Exception:
            op1 = {**FALLBACK_SPECIALIST, "feature": self.f1}
            any_fb = True

        # specialist 2
        raw2 = getattr(
            self.llm.invoke(self.doc2_prompt.invoke({"patient_text": patient_text})),
            "content",
            ""
        )
        try:
            op2 = safe_json_loads(raw2)
        except Exception:
            op2 = {**FALLBACK_SPECIALIST, "feature": self.f2}
            any_fb = True

        # judge/adjudicator
        rawj = getattr(
            self.llm.invoke(self.judge_prompt.invoke({
                "patient_text": patient_text,
                "op1": json.dumps(op1, ensure_ascii=False),
                "op2": json.dumps(op2, ensure_ascii=False),
            })),
            "content",
            ""
        )

        try:
            parsed = safe_json_loads(rawj)
            diag = normalize_diagnosis(parsed.get("diagnosis"))
            if not diag:
                return RunResult("no_disease", rawj, FALLBACK_JUDGE, True)
            return RunResult(diag, rawj, {"diagnosis": diag}, any_fb)
        except Exception:
            return RunResult("no_disease", rawj, FALLBACK_JUDGE, True)
```

15.2 Run

```
llm = build_llm(MODEL_NAME, TEMPERATURE)
mas = TwoSpecialistsMASPipeline(llm, FEATURE_1, FEATURE_2)

rows = []
for idx, row in df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient(row)
    rr = mas.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

mas_pred_df = pd.DataFrame(rows)
mas_pred_df.to_csv(OUT_CSV_SPECIALIST, index=False)
mas_pred_df.head()
```

15.3 Results

```
LABELS = ["no_disease", "disease_present"]

eval_df = (
    df.reset_index(drop=True)
      .reset_index(names="row_index")
      .merge(mas_pred_df[["row_index", "prediction"]],
             on="row_index",
             how="inner")
)

eval_df = eval_df[
    eval_df["disease_present"].isin(LABELS) &
    eval_df["prediction"].isin(LABELS)
].copy()

y_true = eval_df["disease_present"]
y_pred = eval_df["prediction"]

print("=" * 90)
print("Feature-Specialist Protocol — classification_report")
print(classification_report(
    y_true,
    y_pred,
    labels=LABELS,
    target_names=LABELS,
    digits=2
))
```

16 Results Analysis

16.1 Overall Performance (Cleveland)

System	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
Generic Deliberative Protocol (Doctor A + Doctor B + Adjudicator)	0.65	0.66	0.66	0.65
Feature-Specialist Protocol (Specialist + Specialist + Judge)	0.72	0.72	0.71	0.71

16.2 Overall Performance (Pima Indians Diabetes)

System	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
Generic Deliberative Protocol (Doctor A + Doctor B + Adjudicator)	0.68	0.65	0.64	0.64
Feature-Specialist Protocol (Specialist + Specialist + Judge)	0.51	0.66	0.61	0.49

16.3 Class-wise Performance (Cleveland)

System	Class	Precision	Recall	F1-score
Generic Deliberative Protocol	no_disease	0.72	0.60	0.65
	disease_present	0.60	0.72	0.66
Feature-Specialist Protocol	no_disease	0.70	0.82	0.76
	disease_present	0.74	0.59	0.66

16.4 Class-wise Performance (Pima Indians Diabetes)

System	Class	Precision	Recall	F1-score
Generic Deliberative Protocol	no_disease	0.74	0.79	0.77
	disease_present	0.55	0.48	0.51
Feature-Specialist Protocol	no_disease	0.91	0.27	0.42
	disease_present	0.41	0.95	0.57

16.5 Confusion Matrices

16.5.1 Generic Deliberative Protocol

```
LABELS = ["no_disease", "disease_present"]

eval_df = (
    df.reset_index(drop=True)
      .reset_index(names="row_index")
      .merge(generic_pred_df[["row_index", "prediction"]],
             on="row_index",
             how="inner")
)

eval_df = eval_df[
    eval_df["disease_present"].isin(LABELS) &
    eval_df["prediction"].isin(LABELS)
].copy()

y_true = eval_df["disease_present"]
y_pred = eval_df["prediction"]

cm_generic = confusion_matrix(
    y_true,
    y_pred,
    labels=LABELS
)
```

```
cm_generic_df = pd.DataFrame(
    cm_generic,
    index=[f"Actual_{l}" for l in LABELS],
    columns=[f"Pred_{l}" for l in LABELS]
)

print("=" * 90)
print("Generic Deliberative Protocol — Confusion Matrix")
display(cm_generic_df)
```

16.5.2 Feature Specialist Protocol

```
LABELS = ["no_disease", "disease_present"]

eval_df = (
    df.reset_index(drop=True)
      .reset_index(names="row_index")
      .merge(mas_pred_df[["row_index", "prediction"]],
             on="row_index",
             how="inner")
)

eval_df = eval_df[
    eval_df["disease_present"].isin(LABELS) &
    eval_df["prediction"].isin(LABELS)
].copy()

y_true = eval_df["disease_present"]
y_pred = eval_df["prediction"]

cm_specialist = confusion_matrix(
    y_true,
    y_pred,
    labels=LABELS
)

cm_specialist_df = pd.DataFrame(
    cm_specialist,
    index=[f"Actual_{l}" for l in LABELS],
    columns=[f"Pred_{l}" for l in LABELS]
)

print("=" * 90)
print("Feature-Specialist Protocol — Confusion Matrix")
display(cm_specialist_df)
```

17 Ablation Study: Feature-Set Sensitivity of the Feature-Specialist Protocol (Cleveland)

17.1 Motivation

A key concern raised in peer review is that the observed performance differences between the Feature-Specialist (FS) and Generic Deliberative protocols may be an artifact of the specific feature pair chosen for the FS protocol rather than evidence that role decomposition itself acts as an architectural inductive bias. To address this, we conduct a systematic ablation on the UCI Cleveland Heart Disease dataset, evaluating five additional feature-pair conditions that span the range of univariate predictive signal available in the dataset. The original pair (`chest_pain_type` + `num_major_vessels_fluoroscopy`) from the main experiment serves as the reference; results for that pair are already reported in the main results section.

For each condition we use the identical `TwoSpecialistsMASPipeline` and judge architecture from the main experiment—only the two features assigned to the specialists change. This isolates feature selection as the sole independent variable.

17.2 Feature Conditions and Rationale

Condition	Feature 1	Feature 2	Rationale
B — High-signal alternative	`thalassemia_test_result`	`st_depression_exercise_relative_to_rest`	`thal` is consistently the top-ranked predictor in Cleveland feature-importance analyses (logistic regression, random forest); ST depression (`oldpeak`) is a direct physiological marker of myocardial ischaemia during exertion. Neither feature overlaps with the original pair, providing an independent high-signal comparison.
C — Correlated exercise-stress pair	`exercise_induced_angina`	`st_segment_slope_peak_exercise`	Both features are derived from the same exercise stress test protocol and are clinically correlated—angina during exertion and the morphology of the ST-segment response co-occur in ischaemic disease. This tests whether the FS protocol is sensitive to collinear specialist assignments where information is partially redundant.

Condition	Feature 1	Feature 2	Rationale
D — Weak-signal pair	`fasting_blood_sugar_over_120_mg_dL`	`serum_cholesterol_mg_dL`	FBS and cholesterol have consistently low univariate discriminative power in the Cleveland cohort: FBS is elevated in only ~15% of patients with near-equal class distributions; cholesterol shows weak and overlapping distributions across disease/no-disease groups. This condition tests the floor—whether any FS advantage collapses when specialists receive minimally informative features.
E — Mixed-signal pair	`max_heart_rate_bpm`	`thalassemia_test_result`	Pairs a moderate predictor (maximum heart rate, which inversely correlates with CAD severity but with substantial distributional overlap) with the single strongest predictor (`thal`). Tests asymmetric specialist quality—whether a single highly-informative specialist can compensate for a weaker co-specialist within the same FS architecture.
F — Demographic proxy pair	`age_years`	`sex`	Purely demographic features with no direct cardiac measurement. Age and sex are risk-stratification proxies rather than diagnostic markers in isolation. Tests whether the FS architecture degrades toward uninformed prediction when specialists hold only population-level proxy information, establishing a lower bound distinct from the weak-signal clinical pair in Condition D.

```
cleveland_df = pd.read_csv("data/processed_cleveland.csv")

LABELS = ["no_disease", "disease_present"]
print("Shape:", cleveland_df.shape)
print(cleveland_df["disease_present"].value_counts())
cleveland_df.head(2)
```

```
def serialize_patient_cleveland(row):
```

```
    return f"""
Patient information:
- Age: {row['age_years']} years
- Sex: {row['sex']}
- Chest pain type: {row['chest_pain_type']}
- Resting blood pressure: {row['resting_blood_pressure_mmHg']} mmHg
- Serum cholesterol: {row['serum_cholesterol_mg_dL']} mg/dL
- Fasting blood sugar > 120 mg/dL: {row['fasting_blood_sugar_over_120_mg_dL']}
- Resting ECG result: {row['resting_ecg_result']}
- Maximum heart rate achieved: {row['max_heart_rate_bpm']} bpm
- Exercise-induced angina: {row['exercise_induced_angina']}
- ST depression (exercise vs rest): {row['st_depression_exercise_relative_to_rest']}
- ST segment slope (peak exercise): {row['st_segment_slope_peak_exercise']}
- Number of major vessels (fluoroscopy): {row['num_major_vessels_fluoroscopy']}
- Thalassemia test result: {row['thalassemia_test_result']}
"""""

print(serialize_patient_cleveland(cleveland_df.iloc[0]))
```

```
"""ABLATION_CONDITIONS = [
    (
        "B — High-signal alternative",
        "thalassemia_test_result",
        "st_depression_exercise_relative_to_rest",
        "Two strongest non-overlapping predictors; thal is top-ranked in Cleveland; oldpeak is direct
     ischaemia marker.",
    ),
    (
        "C — Correlated exercise-stress pair",
        "exercise_induced_angina",
        "st_segment_slope_peak_exercise",
        "Both from same stress-test protocol and are clinically correlated; tests redundancy
    sensitivity.",
    ),
    (
        "D — Weak-signal pair",
        "fasting_blood_sugar_over_120_mg_dL",
        "serum_cholesterol_mg_dL",
        "Lowest univariate discriminative power in Cleveland; tests floor condition.",
    ),
    (
        "E — Mixed-signal pair",
        "max_heart_rate_bpm",
        "thalassemia_test_result",
        "Moderate predictor paired with strongest predictor; tests asymmetric specialist quality.",
    )
]"""
```

```
"""from sklearn.metrics import accuracy_score, f1_score, classification_report

llm = build_llm(MODEL_NAME, TEMPERATURE)
ablation_results = []
ablation_classwise = []
```

```
for condition_label, f1_name, f2_name, note in ABLATION_CONDITIONS:
    print(f"\nRunning: {condition_label} | {f1_name} + {f2_name}")

    pipeline = TwoSpecialistsMASPipeline(llm, f1_name, f2_name)

    rows = []
    for idx, row in cleveland_df.reset_index(drop=True).iterrows():
        patient_text = serialize_patient_cleveland(row)
        rr = pipeline.run(idx, patient_text)
        rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used
    })

    pred_df = pd.DataFrame(rows)

    eval_df = (
        cleveland_df.reset_index(drop=True)
        .reset_index(names="row_index")
        .merge(pred_df[["row_index", "prediction"]], on="row_index", how="inner")
    )
    eval_df = eval_df[
        eval_df["disease_present"].isin(LABELS) &
        eval_df["prediction"].isin(LABELS)
    ].copy()

    y_true = eval_df["disease_present"]
    y_pred  = eval_df["prediction"]

    acc      = accuracy_score(y_true, y_pred)
    macro_f1 = f1_score(y_true, y_pred, labels=LABELS, average="macro")
    report   = classification_report(y_true, y_pred, labels=LABELS, output_dict=True)

    print(f"  Accuracy: {acc:.3f} | Macro-F1: {macro_f1:.3f}")
    print(f"  | Class           | Precision | Recall | F1   |")
    print(f"  |-----------------|-----------|--------|------|")
    for cls in LABELS:
        r = report[cls]
        print(f"  | {cls:<15} | {r['precision']:.2f}      | {r['recall']:.2f}   | {r['f1-score']:.2f}
     |")

    ablation_results.append({
        "FS feature condition": condition_label,
        "Feature 1": f1_name,
        "Feature 2": f2_name,
        "Accuracy": round(acc, 3),
        "Macro-F1": round(macro_f1, 3),
        "Notes": note,
    })

    for cls in LABELS:
        ablation_classwise.append({
            "FS feature condition": condition_label,
            "Class": cls,
            "Precision": round(report[cls]["precision"], 2),
            "Recall": round(report[cls]["recall"], 2),
            "F1-score": round(report[cls]["f1-score"], 2),
        })
```

```
    pred_df.to_csv(
        f"results/heart_disease/ablation_{condition_label[:1].lower()}_predictions.csv",
        index=False,
    )

ablation_df = pd.DataFrame(ablation_results)
ablation_cw_df = pd.DataFrame(ablation_classwise)
print("\nDone.")"""
```

```
"""pd.set_option("display.max_colwidth", None)

ablation_reference_a = pd.DataFrame([{
    "Dataset": "processed_cleveland.csv",
    "FS feature condition": "A — Original FS pair (main experiment)",
    "Feature 1": "chest_pain_type",
    "Feature 2": "num_major_vessels_fluoroscopy",
    "Accuracy": 0.716,
    "Macro-F1": 0.707,
    "Notes": "Original feature pair from the main FS experiment; shown as the reference condition,
    not rerun in this ablation loop.",
}])

ablation_results_with_reference = pd.concat(
    [ablation_reference_a, ablation_df],
    ignore_index=True,
)

display(ablation_results_with_reference[[
    "Dataset", "FS feature condition", "Feature 1", "Feature 2",
    "Accuracy", "Macro-F1", "Notes"
]])

from pathlib import Path
from sklearn.metrics import precision_recall_fscore_support

ablation_prediction_files = {
    "A — Original FS pair (main experiment)": "results/heart_disease/specialist-predictions.csv",
}
for condition_label, _, _, _ in ABLATION_CONDITIONS:
    ablation_prediction_files[condition_label] = (
        f"results/heart_disease/ablation_{condition_label[:1].lower()}_predictions.csv"
    )

classwise_rows = []
for condition_label, pred_path in ablation_prediction_files.items():
    if not Path(pred_path).exists():
        print(f"Skipping {condition_label}: {pred_path} not found yet.")
        continue

    pred_df = pd.read_csv(pred_path)
    eval_df = (
        cleveland_df.reset_index(drop=True)
        .reset_index(names="row_index")
        .merge(pred_df[["row_index", "prediction"]], on="row_index", how="inner")
```

```
    )
    eval_df = eval_df[
        eval_df["disease_present"].isin(LABELS) &
        eval_df["prediction"].isin(LABELS)
    ].copy()

    precision, recall, f1, support = precision_recall_fscore_support(
        eval_df["disease_present"],
        eval_df["prediction"],
        labels=LABELS,
        zero_division=0,
    )

    for label, p, r, f, s in zip(LABELS, precision, recall, f1, support):
        classwise_rows.append({
            "FS feature condition": condition_label,
            "Class": label,
            "Precision": round(float(p), 2),
            "Recall": round(float(r), 2),
            "F1-score": round(float(f), 2),
            "Support": int(s),
        })

ablation_classwise_df = pd.DataFrame(classwise_rows)
display(ablation_classwise_df)"""
```

17.3 Ablation Results Summary

Note: Conditions B–E are populated from the completed ablation run. Condition A is the original Feature-Specialist pair from the main Cleveland experiment and is included here only as a reference; it is not rerun in the ablation loop. Condition F remains unpopulated unless the demographic-proxy ablation is run separately.

Dataset	FS feature condition	Feature 1	Feature 2	Accuracy	Macro-F1	Notes
processed_cleveland.csv	A — Original FS pair (main experiment)	chest_pain_type	num_major_vessels_fluoroscopy	0.716	0.707	Original feature pair from the main FS experiment; reference condition only.
processed_cleveland.csv	B — High-signal alternative	thalassemia_test_result	st_depression_exercise_relative_to_rest	0.620	0.616	Alternative high-signal pair; produced high recall for disease_present but lower recall for no_disease.
processed_cleveland.csv	C — Correlated exercise-stress pair	exercise_induced_angina	st_segment_slope_peak_exercise	0.693	0.674	Best-performing ablation condition; both features relate to exercise/stress-test response and produced high no_disease recall.
processed_cleveland.csv	D — Weak-signal pair	fasting_blood_sugar_over_120_mg_dL	serum_cholesterol_mg_dL	0.452	0.326	Weak-signal/floor condition; heavily biased toward disease_present with very low no_disease recall.
processed_cleveland.csv	E — Mixed-signal pair	max_heart_rate_bpm	thalassemia_test_result	0.409	0.403	Mixed-signal condition; underperformed relative to the original FS pair and other high-signal alternatives.

17.4 Ablation Class-wise Performance Summary

Note: This table is computed from the saved prediction CSVs. Condition A is read from `results/heart_disease/specialist-predictions.csv`; Conditions B–F are read from their ablation prediction files after the ablation loop has been run.

FS feature condition	Class	Precision	Recall	F1-score	Support
A — Original FS pair (main experiment)	no_disease	0.70	0.82	0.76	164
A — Original FS pair (main experiment)	disease_present	0.74	0.59	0.66	139
B — High-signal alternative	no_disease	0.73	0.47	0.57	164
B — High-signal alternative	disease_present	0.56	0.80	0.66	139
C — Correlated exercise-stress pair	no_disease	0.67	0.87	0.75	164
C — Correlated exercise-stress pair	disease_present	0.76	0.49	0.59	139
D — Weak-signal pair	no_disease	0.38	0.02	0.03	164
D — Weak-signal pair	disease_present	0.45	0.96	0.62	139
E — Mixed-signal pair	no_disease	0.43	0.29	0.34	164
E — Mixed-signal pair	disease_present	0.40	0.55	0.46	139

18 Ablation Study: Feature-Set Sensitivity of the Feature-Specialist Protocol (Pima Indians Diabetes)

18.1 Motivation

A key concern raised in peer review is that the observed performance differences between the Feature-Specialist (FS) and Generic Deliberative protocols may be an artifact of the specific feature pair chosen for the FS protocol rather than evidence that role decomposition itself acts as an architectural inductive bias. To address this, we conduct a systematic ablation on the Pima Indians Diabetes dataset, evaluating five additional feature-pair conditions that span the range of univariate predictive signal available in the dataset. The original pair (`plasma_glucose_mg_dL` + `body_mass_index`) from the main experiment serves as the reference; results for that pair are already reported in the main results section.

For each condition we use the identical `TwoSpecialistsMASPipeline` and judge architecture from the main experiment, only the two features assigned to the specialists change. This isolates feature selection as the sole independent variable.

18.2 Feature Conditions and Rationale

Condition	Feature 1	Feature 2	Rationale
B — High-signal alternative	`age_years`	`diabetes_pedigree_function`	Among the strongest non-overlapping remaining predictors in the Pima dataset. Age captures cumulative metabolic risk, while diabetes pedigree function encodes familial susceptibility. This provides an independent high-signal comparison that does not reuse the original glucose/BMI pair.
C — Correlated adiposity pair	`triceps_skinfold_thickness_mm`	`body_mass_index`	Both variables reflect adiposity/body composition and are partially redundant in the same obesity-related risk pathway. This tests whether the FS protocol is sensitive to correlated specialist assignments with overlapping physiological content.
D — Weak-signal pair	`diastolic_blood_pressure_mmHg`	`triceps_skinfold_thickness_mm`	Blood pressure and skinfold thickness are comparatively weak standalone discriminators in many Pima analyses relative to glucose and BMI. This condition tests the floor, whether any FS advantage collapses when specialists receive minimally informative features.
E — Mixed-signal pair	`plasma_glucose_mg_dL`	`serum_insulin_uU_mL`	Pairs the single strongest predictor in the dataset (plasma glucose) with a noisier, partially missing metabolic correlate (serum insulin). This tests asymmetric specialist quality, whether one highly informative specialist can compensate for a weaker co-specialist within the same FS architecture.

Condition	Feature 1	Feature 2	Rationale
F — Demographic proxy pair	`age_years`	`pregnancies`	These are life-course and reproductive-history proxies rather than direct contemporaneous glycaemic measurements. This tests whether the FS architecture degrades toward population-level risk stratification when both specialists hold only proxy information.

```
pima_df = pd.read_csv("data/processed_diabetes.csv")

PIMA_LABELS = ["no_disease", "disease_present"]
print("Shape:", pima_df.shape)
print(pima_df["disease_present"].value_counts())
pima_df.head(2)
```

```
def serialize_patient_pima(row):
    return f"""
Patient information:
- Age: {row['age_years']}
- Pregnancies: {row['pregnancies']}
- Plasma glucose concentration (2-hour OGTT): {row['plasma_glucose_mg_dL']} mg/dL
- Diastolic blood pressure: {row['diastolic_blood_pressure_mmHg']} mmHg
- Triceps skinfold thickness: {row['triceps_skinfold_thickness_mm']} mm
- 2-hour serum insulin: {row['serum_insulin_uU_mL']} uU/mL
- Body mass index (BMI): {row['body_mass_index']} kg/m^2
- Diabetes pedigree function: {row['diabetes_pedigree_function']}
"""

print(serialize_patient_pima(pima_df.iloc[0]))
```

```
'''PIMA_ABLATION_CONDITIONS = [
    (
        "B — High-signal alternative",
        "age_years",
        "diabetes_pedigree_function",
        "Strong non-overlapping alternative pair in Pima; age captures cumulative risk and pedigree
    captures familial susceptibility.",
    ),
    (
        "C — Correlated adiposity pair",
        "triceps_skinfold_thickness_mm",
        "body_mass_index",
        "Both features reflect adiposity/body composition and test redundancy sensitivity.",
    ),
    (
        "D — Weak-signal pair",
        "diastolic_blood_pressure_mmHg",
```

```
        "triceps_skinfold_thickness_mm",
        "Comparatively weak standalone predictors in Pima; tests floor condition.",
    ),
    (
        "E — Mixed-signal pair",
        "plasma_glucose_mg_dL",
        "serum_insulin_uU_mL",
        "Strongest predictor paired with a noisier metabolic correlate; tests asymmetric specialist
    quality.",
    )
]'''
```

```
'''from sklearn.metrics import accuracy_score, f1_score, classification_report

llm = build_llm(MODEL_NAME, TEMPERATURE)
pima_ablation_results = []
pima_ablation_classwise = []

for condition_label, f1_name, f2_name, note in PIMA_ABLATION_CONDITIONS:
    print(f"\nRunning: {condition_label} | {f1_name} + {f2_name}")

    pipeline = TwoSpecialistsMASPipeline(llm, f1_name, f2_name)

    rows = []
    for idx, row in pima_df.reset_index(drop=True).iterrows():
        patient_text = serialize_patient_pima(row)
        rr = pipeline.run(idx, patient_text)
        rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used
    })

    pred_df = pd.DataFrame(rows)

    eval_df = (
        pima_df.reset_index(drop=True)
        .reset_index(names="row_index")
        .merge(pred_df[["row_index", "prediction"]], on="row_index", how="inner")
    )
    eval_df = eval_df[
        eval_df["disease_present"].isin(PIMA_LABELS) &
        eval_df["prediction"].isin(PIMA_LABELS)
    ].copy()

    y_true = eval_df["disease_present"]
    y_pred = eval_df["prediction"]

    acc = accuracy_score(y_true, y_pred)
    macro_f1 = f1_score(y_true, y_pred, labels=PIMA_LABELS, average="macro")
    report = classification_report(y_true, y_pred, labels=PIMA_LABELS, output_dict=True)

    print(f"  Accuracy: {acc:.3f} | Macro-F1: {macro_f1:.3f}")
    print(f"  | Class           | Precision | Recall | F1   |")
    print(f"  |-----------------|-----------|--------|------|")
    for cls in PIMA_LABELS:
        r = report[cls]
        print(f"  | {cls:<15} | {r['precision']:.2f}      | {r['recall']:.2f}   | {r['f1-score']:.2f}
```

```
      |")

    pima_ablation_results.append({
        "FS feature condition": condition_label,
        "Feature 1": f1_name,
        "Feature 2": f2_name,
        "Accuracy": round(acc, 3),
        "Macro-F1": round(macro_f1, 3),
        "Notes": note,
    })

    for cls in PIMA_LABELS:
        pima_ablation_classwise.append({
            "FS feature condition": condition_label,
            "Class": cls,
            "Precision": round(report[cls]["precision"], 2),
            "Recall": round(report[cls]["recall"], 2),
            "F1-score": round(report[cls]["f1-score"], 2),
        })

    pred_df.to_csv(
        f"results/diabetes/ablation_{condition_label[:1].lower()}_predictions.csv",
        index=False,
    )

pima_ablation_df = pd.DataFrame(pima_ablation_results)
pima_ablation_cw_df = pd.DataFrame(pima_ablation_classwise)
print("\nDone.")'''
```

```
'''pd.set_option("display.max_colwidth", None)

pima_ablation_reference_a = pd.DataFrame([{
    "Dataset": "processed_diabetes.csv",
    "FS feature condition": "A — Original FS pair (main experiment)",
    "Feature 1": "plasma_glucose_mg_dL",
    "Feature 2": "body_mass_index",
    "Accuracy": 0.507,
    "Macro-F1": 0.494,
    "Notes": "Original feature pair from the main FS experiment; shown as the reference condition,
    not rerun in this ablation loop.",
}])

pima_ablation_results_with_reference = pd.concat(
    [pima_ablation_reference_a, pima_ablation_df],
    ignore_index=True,
)

display(pima_ablation_results_with_reference[[
    "Dataset", "FS feature condition", "Feature 1", "Feature 2",
    "Accuracy", "Macro-F1", "Notes"
]])

from pathlib import Path
from sklearn.metrics import precision_recall_fscore_support
```

```
pima_ablation_prediction_files = {
    "A — Original FS pair (main experiment)": "results/diabetes/specialist-predictions.csv",
}
for condition_label, _, _, _ in PIMA_ABLATION_CONDITIONS:
    pima_ablation_prediction_files[condition_label] = (
        f"results/diabetes/ablation_{condition_label[:1].lower()}_predictions.csv"
    )

pima_classwise_rows = []
for condition_label, pred_path in pima_ablation_prediction_files.items():
    if not Path(pred_path).exists():
        print(f"Skipping {condition_label}: {pred_path} not found yet.")
        continue

    pred_df = pd.read_csv(pred_path)
    eval_df = (
        pima_df.reset_index(drop=True)
        .reset_index(names="row_index")
        .merge(pred_df[["row_index", "prediction"]], on="row_index", how="inner")
    )
    eval_df = eval_df[
        eval_df["disease_present"].isin(PIMA_LABELS) &
        eval_df["prediction"].isin(PIMA_LABELS)
    ].copy()

    precision, recall, f1, support = precision_recall_fscore_support(
        eval_df["disease_present"],
        eval_df["prediction"],
        labels=PIMA_LABELS,
        zero_division=0,
    )

    for label, p, r, f, s in zip(PIMA_LABELS, precision, recall, f1, support):
        pima_classwise_rows.append({
            "FS feature condition": condition_label,
            "Class": label,
            "Precision": round(float(p), 2),
            "Recall": round(float(r), 2),
            "F1-score": round(float(f), 2),
            "Support": int(s),
        })

pima_ablation_classwise_df = pd.DataFrame(pima_classwise_rows)
display(pima_ablation_classwise_df)'''
```

18.3 Ablation Results Summary

Note: Conditions B–E are defined here for the Pima Indians Diabetes ablation setup. Condition A is the original Feature-Specialist pair from the main Pima experiment and is included here only as a reference; it is not rerun in the ablation loop. Condition F remains unpopulated unless the demographic-proxy ablation is run separately.

Dataset	FS feature condition	Feature 1	Feature 2	Accuracy	Macro-F1	Notes
processed_diabetes.csv	A — Original FS pair (main experiment)	plasma_glucose_mg_dL	body_mass_index	0.507	0.494	Original feature pair from the main FS experiment; shown as the reference condition and not rerun in the ablation loop.
processed_diabetes.csv	B — High-signal alternative	age_years	diabetes_pedigree_function	0.454	0.430	Strong non-overlapping alternative pair in Pima; age captures cumulative risk and pedigree captures hereditary risk.
processed_diabetes.csv	C — Correlated adiposity pair	triceps_skinfold_thickness_mm	body_mass_index	0.496	0.496	Both features reflect adiposity/body composition and test redundancy sensitivity.
processed_diabetes.csv	D — Weak-signal pair	diastolic_blood_pressure_mmHg	triceps_skinfold_thickness_mm	0.415	0.414	Comparatively weak standalone predictors in Pima; used as a lower-signal/floor condition.
processed_diabetes.csv	E — Mixed-signal pair	plasma_glucose_mg_dL	serum_insulin_uU_mL	0.479	0.469	Strongest predictor paired with a noisier metabolic correlate; tests asymmetric specialist quality.

18.4 Ablation Class-wise Performance Summary

Note: This table is computed from the saved prediction CSVs. Condition A is read from `results/diabetes/specialist-predictions.csv`; Conditions B–F are read from their ablation prediction files after the ablation loop has been run.

FS feature condition	Class	Precision	Recall	F1-score	Support
A — Original FS pair (main experiment)	no_disease	0.91	0.27	0.42	500
A — Original FS pair (main experiment)	disease_present	0.41	0.95	0.57	268
B — High-signal alternative	no_disease	0.87	0.19	0.31	500
B — High-signal alternative	disease_present	0.39	0.75	0.55	268
C — Correlated adiposity pair	no_disease	0.70	0.40	0.51	500
C — Correlated adiposity pair	disease_present	0.38	0.68	0.48	268
D — Weak-signal pair	no_disease	0.61	0.28	0.38	500
D — Weak-signal pair	disease_present	0.33	0.67	0.44	268
E — Mixed-signal pair	no_disease	0.63	0.48	0.54	500
E — Mixed-signal pair	disease_present	0.33	0.49	0.39	268

19 Ablation Study: Model Scale – Qwen2.5 14B (Original GD and FS Configurations)

19.1 Motivation

The main experiment and the prior feature-set ablation studies all use `llama3.1:8b` as the base LLM. A natural question is whether the observed protocol-level differences between the Generic Deliberative (GD) and

Feature-Specialist (FS) architectures are a property of the role decomposition itself, or are an artefact of a specific model scale. To probe this, we rerun the original GD and original FS configurations — unmodified prompts and pipeline architectures — on both the UCI Cleveland Heart Disease dataset and the Pima Indians Diabetes dataset using `qwen2.5:14b`, a 14B-parameter instruction-tuned model, under identical deterministic conditions (temperature = 0).

The only variable that changes across this ablation is the underlying LLM; feature pairs, system prompts, pipeline logic, and evaluation protocols are held constant from the main experiment.

```
from pathlib import Path

cleveland_df = pd.read_csv("data/processed_cleveland.csv")
pima_df = pd.read_csv("data/processed_diabetes.csv")

if "serialize_patient_cleveland" not in globals():
    def serialize_patient_cleveland(row):
        return f"""
Patient information:
- Age: {row['age_years']} years
- Sex: {row['sex']}
- Chest pain type: {row['chest_pain_type']}
- Resting blood pressure: {row['resting_blood_pressure_mmHg']} mmHg
- Serum cholesterol: {row['serum_cholesterol_mg_dL']} mg/dL
- Fasting blood sugar > 120 mg/dL: {row['fasting_blood_sugar_over_120_mg_dL']}
- Resting ECG result: {row['resting_ecg_result']}
- Maximum heart rate achieved: {row['max_heart_rate_bpm']} bpm
- Exercise-induced angina: {row['exercise_induced_angina']}
- ST depression (exercise vs rest): {row['st_depression_exercise_relative_to_rest']}
- ST segment slope (peak exercise): {row['st_segment_slope_peak_exercise']}
- Number of major vessels (fluoroscopy): {row['num_major_vessels_fluoroscopy']}
- Thalassemia test result: {row['thalassemia_test_result']}
"""

if "serialize_patient_pima" not in globals():
    def serialize_patient_pima(row):
        return f"""
Patient information:
- Age: {row['age_years']}
- Pregnancies: {row['pregnancies']}
- Plasma glucose concentration (2-hour OGTT): {row['plasma_glucose_mg_dL']} mg/dL
- Diastolic blood pressure: {row['diastolic_blood_pressure_mmHg']} mmHg
- Triceps skinfold thickness: {row['triceps_skinfold_thickness_mm']} mm
- 2-hour serum insulin: {row['serum_insulin_uU_mL']} uU/mL
- Body mass index (BMI): {row['body_mass_index']} kg/m^2
- Diabetes pedigree function: {row['diabetes_pedigree_function']}
"""

Path("results/heart_disease").mkdir(parents=True, exist_ok=True)
Path("results/diabetes").mkdir(parents=True, exist_ok=True)

MODEL_NAME_14B = "qwen2.5:14b"

llm_14b = build_llm(MODEL_NAME_14B, TEMPERATURE)
```

```
# Generic Deliberative Protocol — Cleveland
```

```
gd_14b_cleveland = GenericDeliberativeMASPipeline(llm_14b)

rows = []
for idx, row in cleveland_df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient_cleveland(row)
    rr = gd_14b_cleveland.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

gd_14b_cleveland_pred_df = pd.DataFrame(rows)
gd_14b_cleveland_pred_df.to_csv("results/heart_disease/14b_gd_predictions.csv", index=False)
print("Generic Deliberative (qwen2.5:14b) — Cleveland done.")
gd_14b_cleveland_pred_df.head()
```

```
# Feature-Specialist Protocol — Cleveland (original pair: chest_pain_type +
    num_major_vessels_fluoroscopy)
fs_14b_cleveland = TwoSpecialistsMASPipeline(llm_14b, "chest_pain_type", "
    num_major_vessels_fluoroscopy")

rows = []
for idx, row in cleveland_df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient_cleveland(row)
    rr = fs_14b_cleveland.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

fs_14b_cleveland_pred_df = pd.DataFrame(rows)
fs_14b_cleveland_pred_df.to_csv("results/heart_disease/14b_fs_predictions.csv", index=False)
print("Feature-Specialist (qwen2.5:14b) — Cleveland done.")
fs_14b_cleveland_pred_df.head()
```

```
# Generic Deliberative Protocol — Pima Indians
gd_14b_pima = GenericDeliberativeMASPipeline(llm_14b)

rows = []
for idx, row in pima_df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient_pima(row)
    rr = gd_14b_pima.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

gd_14b_pima_pred_df = pd.DataFrame(rows)
gd_14b_pima_pred_df.to_csv("results/diabetes/14b_gd_predictions.csv", index=False)
print("Generic Deliberative (qwen2.5:14b) — Pima Indians done.")
gd_14b_pima_pred_df.head()
```

```
# Feature-Specialist Protocol — Pima Indians (original pair: plasma_glucose_mg_dL + body_mass_index)
fs_14b_pima = TwoSpecialistsMASPipeline(llm_14b, "plasma_glucose_mg_dL", "body_mass_index")

rows = []
for idx, row in pima_df.reset_index(drop=True).iterrows():
    patient_text = serialize_patient_pima(row)
    rr = fs_14b_pima.run(idx, patient_text)
    rows.append({"row_index": idx, "prediction": rr.diagnosis, "fallback_used": rr.fallback_used})

fs_14b_pima_pred_df = pd.DataFrame(rows)
```

```
fs_14b_pima_pred_df.to_csv("results/diabetes/14b_fs_predictions.csv", index=False)
print("Feature-Specialist (qwen2.5:14b) — Pima Indians done.")
fs_14b_pima_pred_df.head()
```

```
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix

LABELS = ["no_disease", "disease_present"]

_runs_14b = [
    ("Generic Deliberative (qwen2.5:14b)", "Cleveland",     cleveland_df, gd_14b_cleveland_pred_df),
    ("Feature-Specialist (qwen2.5:14b)",   "Cleveland",     cleveland_df, fs_14b_cleveland_pred_df),
    ("Generic Deliberative (qwen2.5:14b)", "Pima Indians",  pima_df,      gd_14b_pima_pred_df),
    ("Feature-Specialist (qwen2.5:14b)",   "Pima Indians",  pima_df,      fs_14b_pima_pred_df),
]

for protocol, dataset, source_df, pred_df in _runs_14b:
    eval_df = (
        source_df.reset_index(drop=True)
        .reset_index(names="row_index")
        .merge(pred_df[["row_index", "prediction"]], on="row_index", how="inner")
    )
    eval_df = eval_df[
        eval_df["disease_present"].isin(LABELS) &
        eval_df["prediction"].isin(LABELS)
    ].copy()
    y_true = eval_df["disease_present"]
    y_pred  = eval_df["prediction"]

    print("=" * 90)
    print(f"{protocol} — {dataset} — classification_report")
    print(classification_report(y_true, y_pred, labels=LABELS, target_names=LABELS, digits=2))

    cm = confusion_matrix(y_true, y_pred, labels=LABELS)
    cm_df = pd.DataFrame(
        cm,
        index=[f"Actual_{l}" for l in LABELS],
        columns=[f"Pred_{l}" for l in LABELS],
    )
    print("Confusion Matrix:")
    display(cm_df)
    print()
```

19.2 Model-Scale Ablation Results (qwen2.5:14b)

19.3 Overall Performance (Cleveland)

System	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
Generic Deliberative Protocol — qwen2.5:14b (Doctor A + Doctor B + Adjudicator)	0.72	0.74	0.70	0.70
Feature-Specialist Protocol — qwen2.5:14b (Specialist + Specialist + Judge)	0.56	0.58	0.53	0.46

19.4 Overall Performance (Pima Indians Diabetes)

System	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
Generic Deliberative Protocol — qwen2.5:14b (Doctor A + Doctor B + Adjudicator)	0.73	0.71	0.67	0.67
Feature-Specialist Protocol — qwen2.5:14b (Specialist + Specialist + Judge)	0.69	0.70	0.72	0.69

19.5 Class-wise Performance (Cleveland)

System	Class	Precision	Recall	F1-score
Generic Deliberative Protocol — qwen2.5:14b	no_disease	0.69	0.89	0.77
	disease_present	0.80	0.52	0.63
Feature-Specialist Protocol — qwen2.5:14b	no_disease	0.56	0.92	0.70
	disease_present	0.61	0.14	0.23

19.6 Class-wise Performance (Pima Indians Diabetes)

System	Class	Precision	Recall	F1-score
Generic Deliberative Protocol — qwen2.5:14b	no_disease	0.75	0.89	0.81
	disease_present	0.68	0.44	0.54
Feature-Specialist Protocol — qwen2.5:14b	no_disease	0.86	0.63	0.73
	disease_present	0.54	0.81	0.65